Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Пермский национальный исследовательский
политехнический университет»**

Электротехнический факультет
Кафедра «Информационные технологии и автоматизированные системы»
направление подготовки: 09.03.01– «Информатика и вычислительная техника»

Лабораторная работа № 2
по дисциплине
«Информатика»
на тему
«Использование основных операторов языка С»
Вариант 12

Выполнил студент гр. ИКС-26-1б
Захлевных Иван Алексеевич

Проверил:
Духанин Алексей Владимирович

(оценка) (подпись)

(дата)

г. Пермь, 2026

ВВЕДЕНИЕ

Вариант: 12

Цель: получение навыков в выборе и использовании операторов C++; знакомство с итерационными процессами.

Задачи:

1. Написать программу решения задачи в соответствии с требованиями.

2. Получить ответ - результат решения для варианта 12.

1 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

С поставленными задачами приведены следующие методические указания:

- при определении суммы членов ряда следует использовать рекуррентную формулу для получения следующего члена ряда;

- при составлении программы считать, что точность достигнута, если $a_n < \varepsilon$.

Поставленная задача, соответствующая варианту: найти сумму ряда с точностью $\varepsilon=10^{-4}$, общий член которого

$$a_n = \frac{2^n n!}{n^n}$$

2 ХОД ВЫПОЛНЕНИЯ РАБОТЫ

2.1 Перевод в рекуррентную форму

Соответственно методическим указаниям следует перевести выражение ряда в рекуррентную форму. Для этого найдём отношение `n + 1`-го члена ряда к `n`-му:

$$\frac{a_{n+1}}{a_n}=\frac{2^{n+1}\times(n+1)!\times n^n}{(n+1)^{n+1}\times 2^n\times n!}=\frac{2\times(n+1)!\times n^n}{(n+1)^{n+1}\times n!}=\frac{2\times(n+1)\times n^n}{(n+1)^{n+1}}=\frac{2\times n^n}{(n+1)^n}=2\times\left(\frac{n}{n+1}\right)^n$$

Таким образом, член `n + 1` можно получить умножением `n`-го члена на $2\times\left(\frac{n}{n+1}\right)^n$

2.2 Составление кода программы

Функция подсчёта суммы ряда оформлена как template-функция, с `T` ограниченным концептом `std::floating_point` для экспериментального подсчёта как с `float` так и с `double`. Сама функция использует `constexpr` константу `eps` для ограничения цикла подсчёта суммы, а также ведёт учёт текущего `n` для установления индекса конца ряда для заданного типа. Нахождение `n`-ого члена производится рекурсивно, посредством умножения предыдущего члена на значение, полученное ранее. Первым значением является $a_1=\frac{2^1\times 1!}{1^1}=\frac{2}{1}=2$, оно и используется для инициализации переменной, отвечающей за прошлый член. В одном месте используется `static_cast<T>`, позволяющий перевести целочисленное деление `n` (типа `std::uintmax_t`) в вещественное деление для соответствующего типа. В качестве библиотеки вывода используется fmtlib за свой удобный API, стандартизированный в C++23 и рекордную производительность.

Весь исходный код приведён в приложении А. Схема алгоритма функции, вычисляющей сумму показана на рисунке 1.

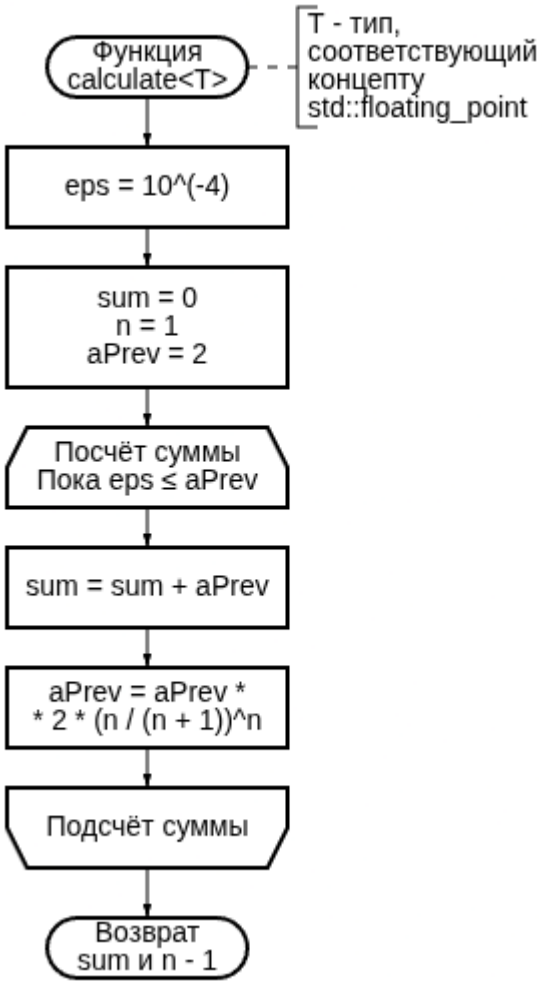


Рисунок 1 - схема алгоритма функции подсчёта

ЗАКЛЮЧЕНИЕ

По результатам исполнения программы (приложение Б) значения и для `float` и для `double` оказались равны в пределах погрешности и примерно равны 12.94856. Последний `n` для заданной погрешности - 38. В ходе работы все задачи были выполнены а цели достигнуты.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 7.32-2017. Межгосударственный стандарт. Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления

ПРИЛОЖЕНИЕ А

Исходный код

```
#include <fmt/base.h>

#include <concepts>
#include <cstdint>
#include <cmath>


template <std::floating_point T>
[[nodiscard]] std::pair<T, std::uintmax_t> calculate() noexcept {
	constexpr T eps = 1.e-4;

	T sum = 0.;
	std::uintmax_t n = 1u;
	for (T aPrev = 2.; eps <= aPrev; ++n) {
		sum += aPrev;

		aPrev *= 2 * std::pow(static_cast<T>(n) / (n + 1u), n);
	}

	return { sum, n - 1u };
}

int main() {
	auto [sumF, nF] = calculate<float>();
	auto [sumD, nD] = calculate<double>();

	fmt::println(
		"float:\n"
		"  sum: {}; last n: {}\n"
		"double:\n"
		"  sum: {}; last n: {}",
		sumF, nF,
		sumD, nD
	);

	return 0;
}
```

ПРИЛОЖЕНИЕ Б

Вывод программы

```
float:
  sum: 12.948564; last n: 38
double:
  sum: 12.94856056092186; last n: 38
```